

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

16. Instancias

deriving

```
data Mes = Ene | Feb | Mar | Abr | May | Jun
         | Jul | Ago | Sep | Oct | Nov | Dic
         deriving (Eq, Ord, Enum, Bounded, Show, Read)
```

- **deriving** hace que Haskell implemente por nosotros lo necesario para que nuestro tipo sea una instancia de las clases especificadas.
- `Abr == Jul` $\rightsquigarrow$ `False` (porque `Eq` ha sido derivado)
- `Abr <= Jul` $\rightsquigarrow$ `True` (porque `Ord` ha sido derivado)
- `succ Feb` $\rightsquigarrow$ `Mar` (porque `Enum` ha sido derivado)
- `[Mar .. Jun]` $\rightsquigarrow$ `[Mar, Abr, May, Jun]` (porque `Enum` ha sido derivado)
- `minBound :: Mes` $\rightsquigarrow$ `Ene` (porque `Bounded` ha sido derivado)
- `show Sep` $\rightsquigarrow$ `"Sep"` (porque `Show` ha sido derivado)
- `read "Sep" :: Mes` $\rightsquigarrow$ `Sep` (porque `Read` ha sido derivado)
- Clases que se pueden derivar:
 - `Eq`
 - `Ord` (requiere `Eq`)
 - `Show`
 - `Read`
 - `Enum` (solo para constructores sin argumentos)
 - `Bounded` (solo para constructores sin argumentos, o para un único constructor)
 - En todos los casos, los tipos de los argumentos tienen que ser también de esa clase.

- Con un tipo con argumentos:

```
data Intervalo a = Intervalo a a
                  deriving (Eq, Ord, Show)
```

- Haskell deriva **Eq** como “mismo constructor e iguales argumentos (según su propia igualdad)”.
- Haskell deriva **Ord** fijándose primero en el orden de los constructores en la definición del tipo y, luego, con un orden lexicográfico sobre los argumentos.
- Haskell deriva **Show** como un reflejo exacto de la sintaxis de la expresión.
- `Intervalo Ene Mar == Intervalo Ene Mar` $\rightsquigarrow$ `True`
- `Intervalo 1 9 < Intervalo 2 3` $\rightsquigarrow$ `True`
- `Left 5 < Right 4` $\rightsquigarrow$ `True`
- `show (Intervalo 'a' 'z')` $\rightsquigarrow$ `"Intervalo 'a' 'z'"`
- `Intervalo id id == Intervalo id id` $\rightsquigarrow$ error: las funciones no son de clase **Eq**

Instancias explícitas

- Fechas en formato día/mes/año:
 - el orden derivado compararía primero los días.

```
data Fecha = Fecha Int Int Int    -- día, mes, año
          deriving Eq

instance Ord Fecha where
    Fecha d1 m1 a1 <= Fecha d2 m2 a2 = (a1, m1, d1) <= (a2, m2, d2)
```

- Conjuntos representados como listas:
 - no importan ni el orden de los elementos en la lista ni las repeticiones;
 - la inclusión de conjuntos no es orden total.

```
data Conjunto a = Conjunto [a]

instance Eq a => Eq (Conjunto a) where
    Conjunto xs == Conjunto ys = all (`elem` ys) xs && all (`elem` xs) ys

esSub :: Eq a => Conjunto a -> Conjunto a -> Bool
esSub (Conjunto xs) (Conjunto ys) = all (`elem` ys) xs
```

- Dos requisitos:
 - **Ord** tiene que ser coherente con **Eq**: `x <= y && y <= x  $\iff$  x == y`.
 - `<=` tiene que implementar un orden total: `x <= y || y <= x`.

Ejercicios

1. Volviendo a los intervalos, tiene sentido decir que todos los intervalos vacíos son iguales, es decir, $(3, 2) = (9, 1)$. Define la manera apropiada en que `Intervalo` es una instancia de `Eq` según este criterio. Mira si la definición de la instancia de `Ord` derivada automáticamente debe ser modificada para mantener la coherencia con la nueva `Eq`.
2. Define un tipo de datos `Fraccion` donde cada valor viene dado por dos números enteros (numerador y denominador). Decláralo como instancia de `Eq` y de `Ord`. Busca información sobre la clase `Num` y haz que `Fraccion` sea también instancia suya. La posibilidad de que los denominadores sean 0 es un problema que, de momento, no sabemos resolver: asume que nunca lo son.
3. Define un tipo `Estacion` con las cuatro estaciones del año. Busca información sobre la clase `Enum` y haz que `Estacion` sea instancia de esa clase de forma cíclica, es decir, consiguiendo que `succ Invierno` sea `Primavera` y `pred Primavera` sea `Invierno`.